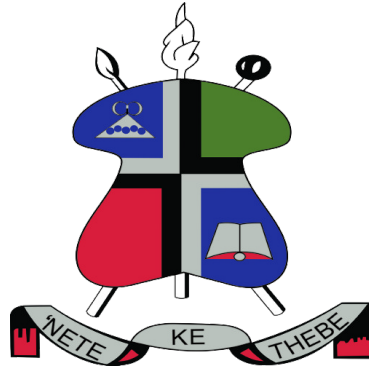


SECURITY OPERATIONS CENTER ANALYST TRIAGE AI ASSISTANT



A final year project submitted by

Kabelo Simon Thesele, 202101668

Napo Julius Qheku, 202101679

In partial fulfilment to the requirements for

B.Sc. Computer Science

To the

Department of Mathematics and Computer Science

Faculty of Science and Technology

National University of Lesotho

April 2026

Supervisor: Mr. Lekuba Ntoane

Acknowledgements

We express sincere gratitude to our supervisor, Mr. L. Ntoane, for his invaluable guidance, constructive feedback, and continuous support throughout this project. To the Department of Mathematics and Computer Science at the National University of Lesotho, we thank you for providing the necessary resources, facilities, and knowledge that made this project possible.

Special recognition goes to Turcotte et al. (2025) for their foundational work on the AACT framework, which served as the cornerstone of this project. A heartfelt thank you to our peers for their encouragement during the challenging phases of this work.

Abstract

Modern Security Operations Centers face an overwhelming challenge of processing thousands to hundreds of thousands of security alerts daily, with up to 95% being false positives. This alert overload leads to analyst fatigue, missed threats, and inefficient operations. This project develops an intelligent triage assistant that reproduces and extends the Automated Alert Classification and Triage (AACT) framework to improve alert prioritization.

The system implements binary classification models distinguishing between malicious and benign alerts using both traditional machine learning (Gradient Boosting with XGBoost) and deep learning approaches (CNN and LSTM networks). Models were trained and evaluated using open cybersecurity datasets (CICIDS2017 and UNSW-NB15), with performance measured through accuracy, precision, recall, and F1-score metrics. Model interpretability was assessed using SHAP values to ensure transparency.

A web-based dashboard prototype was developed using Flask and ReactJS to provide analysts with real-time alert prioritization and visualization. The system incorporates feedback mechanisms for continuous model improvement, aligning with human-on-the-loop decision-making philosophy.

Results demonstrate that the extended deep learning models successfully capture temporal and sequential patterns, achieving significant workload reduction for SOC analysts. The project contributes a practical, explainable, and extensible AI assistant for modern cybersecurity operations.

Contents

Acknowledgements	i
Abstract	ii
1 Introduction	1
1.1 Problem Statement	1

1.2	Aims and Objectives	2
1.3	Scope and Limitations	3
1.4	Methodology	4
2	Literature Review	6
2.1	Introduction	6
2.2	Security Operations Centers and Alert Fatigue	6
2.3	AI Assistants in Cybersecurity	6
2.4	Automated Alert Classification and Triage (AACT)	7
2.5	Machine Learning Methods for Alert Classification	7
2.5.1	Extreme Gradient Boosting (XGBoost)	7
2.5.2	Convolutional Neural Networks for Sequence Modelling	7
2.5.3	Comparison of Existing Approaches	7
2.6	Explainability in Machine Learning	8
2.7	Research Gap and Project Contribution	8
2.8	Tools and Technologies	9
3	Requirements, Design, and Methodology	10
3.1	Detailed Problem Statement	10
3.2	System Requirements	10
3.2.1	Functional Requirements	10
3.2.2	Non-Functional Requirements	12
3.3	Use Case Diagram for SOC Analyst Triage Process	15
4	System Design and Implementation	16
4.1	Design	16
4.1.1	System Architecture	16
4.1.2	Machine Learning Pipeline	17
4.1.3	Database Schema	18
4.1.4	Design Justification	18
4.2	Implementation	19
4.2.1	Machine Learning Implementation	19
4.2.2	XGBoost Implementation	21
4.2.3	The CNN Implementation	23
4.2.4	SHAP Explainability	26
5	Testing and Evaluation	28
5.1	Testing Strategy	28
5.2	Unit Testing	28
5.3	Integration Testing	29

5.4	System Testing	29
5.5	Test Coverage and Summary	30
5.6	Model Evaluation	30
References		36

Chapter 1: Introduction

1.1 Problem Statement

“A Security Operations Center (SOC) is a team of security professionals who monitor an organization’s IT infrastructure, ranging from cloud applications to networked devices. They detect and respond to cybersecurity threats in real time” [1]. Modern Security Operations Centres (SOCs) face an escalating crisis of scale. Analysts are expected to process anywhere between 10,000 and 200,000 security alerts every single day, yet studies consistently show that up to 95% of these alerts are false positives. The human cost of this volume is well documented: analysts suffer from alert fatigue, genuine threats are buried beneath noise, incident response is delayed, and staff turnover rates remain persistently high. The challenge is not merely volume but the absence of reliable context and adaptive prioritization [2].

The reliance on static, rule-based prioritization mechanisms in traditional SOC’s creates a significant operational gap, as these systems cannot adapt to the evolving tactics of attackers or the changing behavioral patterns within a network environment. According to Malatji [3], even with augmented intelligence frameworks designed for human–artificial intelligence teaming in cybersecurity, analysts remain overwhelmed as existing automated techniques fail to capture the dynamic relationship between an alert and its appropriate triage action. The persistence of alert fatigue suggests that static features, such as a specific source IP address, are poor predictors of risk because their relationship with malicious activity fluctuates over time — where the same indicator may signal malicious activity in one context but benign behavior in another, causing traditional models to generate both high false-positive rates and missed detections.

The effectiveness of machine learning approaches in Security Operations Centers is often constrained by their inability to generalize across the heterogeneous and evolving nature of real-world network environments. Femi-Oyewole et al. [4] demonstrate that while ensemble machine learning methods show promise for identifying real-time threats, their performance is heavily dependent on the quality and representativeness of training data, which becomes problematic when models encounter novel attack patterns or previously unseen network behaviors. This limitation is particularly acute in managed SOC settings, where a model trained on one customer’s environment may fail to accurately triage alerts for another organization with different infrastructure, security controls, and baseline activity patterns, necessitating frequent retraining and customization that

undermines operational scalability.

Despite advances in machine learning for alert classification, existing approaches either lack temporal awareness or sufficient interpretability for operational SOC deployment. This project addresses this gap by integrating explainable ensemble learning with temporal deep learning within a human-on-the-loop triage framework.

1.2 Aims and Objectives

Aim

The aim of this study is to design, implement, and evaluate an intelligent AI-driven triage assistant capable of reproducing and extending the Automated Alert Classification and Triage (AACT) framework to enhance alert prioritization within Security Operations Centers. The system seeks to improve the efficiency and reliability of binary classification of security alerts by distinguishing malicious events from benign activity using both ensemble machine learning and deep learning approaches.

To achieve this, the project will integrate Gradient Boosting (XGBoost) for structured alert classification and temporal deep learning architectures, including Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks, to capture sequential patterns in security event streams. The assistant is designed to support human analysts through explainable predictions, interactive visualization, and continuous feedback incorporation, thereby aligning automated decision-making with the human-on-the-loop operational model.

Ultimately, the system aims to reduce alert fatigue, improve detection accuracy, and provide a scalable and adaptable decision-support tool capable of functioning in dynamic cybersecurity environments. Through comparative evaluation and prototype deployment, the study seeks to demonstrate the feasibility of combining explainability, temporal modeling, and analyst feedback within a unified SOC triage framework.

Objectives

1. **Replication:** Reproduce the AACT workflow using the Gradient Boosting classifier (XGBoost) for baseline performance.
2. **Extension:** Implement deep learning models (CNN and LSTM) to capture temporal and sequential alert patterns.
3. **Comparative Evaluation:** Compare performance between Gradient Boosting and deep learning models using accuracy, recall, precision, and F1-score.

4. **Interpretability:** Assess model explainability using SHAP scores to visualize feature contributions and support analyst trust in predictions.
5. **Operationalization:** Design and implement a web-based dashboard for analysts to visualize prioritized alerts and interact with model predictions.
6. **Integration:** Create feedback mechanisms for continuous model improvement based on analyst input.

1.3 Scope and Limitations

Scope

This study focuses specifically on the problem of automated alert triage within Security Operations Centers, conceptualizing alert prioritization as a supervised classification task informed by analyst behavior. Consistent with research on AI-assisted cybersecurity decision support, the system is designed to augment human analysts rather than replace them, aligning with the human-on-the-loop paradigm emphasized in contemporary SOC research.

The project reproduces the core principles of the Automated Alert Classification and Triage framework by modelling alert prioritization using structured security alert data and analyst-driven outcomes. Building upon prior work demonstrating the effectiveness of ensemble learning for structured cybersecurity data, the study implements a Gradient Boosting model (XGBoost) as a baseline classifier. To address limitations identified in traditional machine learning approaches regarding temporal awareness, the system is extended through the integration of deep learning architectures, specifically Convolutional Neural Networks and Long Short-Term Memory networks, to capture sequential and time-dependent patterns in alert streams.

Model development and evaluation are conducted using publicly available cybersecurity datasets, including CICIDS2017 and UNSW-NB15, which provide labelled network traffic and attack scenarios suitable for supervised learning experimentation. Performance is assessed using established classification metrics relevant to SOC environments, including precision, recall, and F1-score, reflecting the operational importance of minimizing false positives and false negatives.

To demonstrate practical applicability, the project includes the development of a prototype web-based interface that presents prioritized alerts, prediction confidence, and explainability outputs. This interface illustrates how machine learning predictions can be integrated into analyst workflows as a decision-support mechanism rather than an autonomous response system.

Accordingly, the study concentrates on alert classification, prioritization, explainability, and analyst interaction, rather than broader cybersecurity functions such as intrusion detection system design, automated response orchestration, or full-scale SOC infrastructure deployment.

Limitations

Model training relies on publicly available datasets that may not fully capture the diversity or evolving characteristics of real-world enterprise environments. The experimental setting reflects controlled evaluation conditions rather than large-scale operational deployment. Furthermore, the system performs binary classification, simplifying the complexity of real SOC triage where alerts may require multi-stage investigation and contextual correlation. This abstraction is necessary to enable controlled model evaluation but does not encompass the full decision-making process observed in operational cybersecurity practice.

1.4 Methodology

The development of the SOC Alert Triage AI Assistant required a methodology capable of supporting two closely related activities: the iterative construction and refinement of machine learning models, and the incremental development of a functional software system. These activities were inherently interdependent, as early modelling results directly influenced subsequent design decisions, including the selection of model architectures and the features implemented within the analyst dashboard. As a result, a purely linear (waterfall) approach was unsuitable, as it does not allow for continuous feedback between modelling and system development.

To address this challenge, **Incremental Prototyping** was selected as the guiding methodology. This approach enables the system to be developed through a sequence of working increments, each producing a functional artefact that can be evaluated before further development. This structure aligns naturally with machine learning workflows, where each modelling phase produces measurable outputs that inform subsequent improvements.

Within this framework, the project was organized into four sequential but interdependent increments:

1. **Baseline Model** — Reproduction of the AACT XGBoost classifier on the UNSW-NB15 dataset, establishing a performance benchmark.
2. **Deep Learning Extension** — Design, training, and evaluation of the CNN architecture, followed by comparison with the XGBoost baseline.
3. **Ensemble and Explainability** — Development of the weighted ensemble model and

integration of SHAP-based explanations into the prediction pipeline.

4. **Dashboard Prototype** — Implementation of the full-stack web application, integrating all models and workflows into an interactive analyst-facing interface.

The first increment focused on establishing a baseline by reproducing the AACT XGBoost classifier on the UNSW-NB15 dataset. This phase provided a validated performance benchmark and initial insights into feature importance, which informed subsequent modelling decisions.

Building on this foundation, the second increment introduced a Convolutional Neural Network (CNN) as a deep learning extension. The evaluation of multiple CNN architectures under the same data conditions enabled a direct comparison with the baseline, revealing complementary strengths between the two approaches, particularly in recall performance.

These findings directly motivated the third increment, in which the XGBoost and CNN models were combined into a weighted ensemble. The decision to ensemble was not predetermined, but emerged from the observed performance characteristics and SHAP-based feature correlation analysis, which indicated that the models captured different aspects of the data. In this phase, explainability was also integrated through SHAP, enabling the system to provide interpretable predictions.

Finally, the fourth increment translated these modelling capabilities into a functional system through the development of a full-stack dashboard. The design of the dashboard was directly informed by the outputs of the previous increments, particularly the need to display prediction confidence, feature importance explanations, and analyst feedback mechanisms.

In this way, each increment produced a functional and evaluable artefact, while simultaneously shaping the direction of subsequent development. This progression reflects the core strength of Incremental Prototyping in supporting projects where system design and model performance evolve together.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the body of knowledge that underpins the design of the proposed SOC Alert Triage AI Assistant. It begins by examining the operational context of Security Operations Centers (SOCs) and the problem of alert fatigue, which motivates the need for automation. It then explores existing approaches to alert classification, progressing from rule-based systems to machine learning models, before examining the AACT framework as the primary reference architecture. Finally, the chapter discusses explainability and identifies the research gap addressed by this project.

2.2 Security Operations Centers and Alert Fatigue

Security Operations Centers (SOCs) are responsible for the continuous monitoring and protection of organizational information systems. Within this environment, alert triage has been identified as the most time-critical and cognitively demanding task [2]. However, SOC analysts are often overwhelmed by extremely high volumes of alerts, many of which are false positives. This leads to *alert fatigue*, a condition in which analysts become desensitized to alerts due to repeated exposure. As a result, genuine threats may be overlooked, posing a significant operational risk.

This limitation highlights the need for automated support systems capable of assisting analysts in prioritizing alerts more effectively.

2.3 AI Assistants in Cybersecurity

To address the challenges associated with alert fatigue, AI-based assistants have been introduced as decision-support tools within SOC environments. These systems use machine learning to learn from historical triage data and predict the priority of new alerts. Importantly, such systems follow a *human-on-the-loop* paradigm, where automation supports decision-making while analysts retain final control. This ensures that efficiency gains do not come at the cost of oversight or accountability. Amongst the various approaches proposed, the Automated Alert Classification and Triage (AACT) framework represents one of the most relevant and influential solutions [1, 2].

2.4 Automated Alert Classification and Triage (AACT)

The Automated Alert Classification and Triage (AACT) framework, proposed by Turcotte et al. (2025), models alert prioritization as a supervised learning problem. In this approach, labels are derived from historical analyst decisions, enabling the system to learn how analysts prioritize alerts in practice. AACT employs XGBoost as its core classification model and incorporates both static alert features and contextual information. Evaluation results demonstrated a reduction of over 60% in analyst workload, highlighting the effectiveness of machine learning in SOC environments. While AACT provides a strong baseline, it primarily relies on structured feature-based learning and does not fully capture temporal relationships within alert sequences. This limitation motivates the exploration of alternative models.

2.5 Machine Learning Methods for Alert Classification

2.5.1 Extreme Gradient Boosting (XGBoost)

XGBoost is a gradient boosting algorithm that improves predictive performance by combining multiple decision trees. Each tree is trained to correct the errors of the previous one, allowing the model to progressively refine its predictions. This approach is particularly effective in SOC environments due to its ability to handle noisy data, missing values, and class imbalance. As a result, XGBoost forms a strong baseline for alert classification systems, including the AACT framework [5].

2.5.2 Convolutional Neural Networks for Sequence Modelling

While XGBoost treats alerts as independent observations, many cyber-attacks occur as sequences of related events. Convolutional Neural Networks (CNNs) address this limitation by capturing local patterns within sequential data. Using 1D convolutions, CNNs can automatically learn temporal relationships in network traffic, potentially identifying attack patterns that feature-based models may miss. However, this increased modelling capability comes at the cost of reduced interpretability, which is a critical requirement in SOC environments.

2.5.3 Comparison of Existing Approaches

The following table summarizes the strengths and limitations of existing SOC triage approaches.

Table 2.1: Comparison of Existing SOC Triage Approaches

Approach	Strengths	Limitations	Relevance to Project
XGBoost	High accuracy, handles imbalance	Limited temporal modeling; treats alerts as independent	Baseline replication of AACT
CNN	Automatic features, temporal patterns	Less interpretable than tree models; requires sequence formatting	Extension for pattern detection
Rule-based	Transparent; no training required; deterministic	Cannot adapt to new threats; high false-positive rates; brittle	Baseline for methodology comparison

2.6 Explainability in Machine Learning

In SOC environments, model transparency is essential for analyst trust and operational adoption. SHAP (SHapley Additive exPlanations) provides a principled method for explaining model predictions by assigning contribution values to each feature. This allows analysts to understand why a prediction was made, enabling validation and supporting informed decision-making.

2.7 Research Gap and Project Contribution

The foregoing review reveals three limitations in existing alert triage research: the insufficiency of static feature-based models for capturing temporal attack patterns; the limited interpretability of high-performing deep learning models; and inadequate integration of analyst feedback into continuous model improvement pipelines. This project addresses these limitations by combining XGBoost and CNN within a weighted ensemble model, where the predictions of both models are aggregated to produce a final classification. This approach leverages the complementary strengths of each model while improving overall performance.

2.8 Tools and Technologies

Implementation uses Python, XGBoost for gradient boosting, TensorFlow/Keras for deep learning, and SHAP for explainability. The web interface employs Flask for the backend API and ReactJS for the frontend visualization.

Summary

This chapter reviewed the key concepts and approaches relevant to the development of the SOC Alert Triage AI Assistant, focusing on machine learning techniques for alert classification within SOC environments. XGBoost was identified as a strong baseline for structured data, while Convolutional Neural Networks were explored to address its limitations in capturing complex feature relationships. The complementary strengths of these models motivated the adoption of an ensemble approach, and the importance of explainability was addressed through SHAP to ensure transparent and analyst-aligned decision-making. Collectively, the literature highlights the need for a system that integrates multiple modelling techniques, supports interpretability, and incorporates analyst feedback. Building on this foundation, Chapter 3 formalizes the problem context and defines the functional and non-functional requirements that guide the system design.

Chapter 3: Requirements, Design, and Methodology

3.1 Detailed Problem Statement

Security Operations Centers (SOCs) generate extremely high volumes of alerts, often exceeding 200,000 per day in large organizations [2], making manual review impractical. While automated solutions exist, many fail to adapt to evolving threats or provide sufficient context for effective decision-making. Rule-based systems produce high false-positive rates, and some machine learning models do not capture the evolving nature of attacks. In addition, limited explainability reduces analyst trust in automated predictions [3].

These challenges highlight the need for a more effective approach that combines multiple machine learning techniques, provides interpretable outputs, supports analyst involvement, and scales to handle large alert volumes.

3.2 System Requirements

Requirements Engineering

The requirements gathering phase aimed to identify SOC challenges addressable by machine learning-based triage, define measurable system behaviors, and ensure feasibility within a final-year prototype. Requirements were derived from the AACT framework, which informed alert classification workflows, and the UNSW-NB15 dataset, which constrained feature selection and preprocessing. Aligned with an incremental prototyping approach, requirements were refined progressively during development and classified into functional requirements, defining system behavior, and non-functional requirements, specifying quality attributes such as performance, usability, and reliability.

3.2.1 Functional Requirements

Table 3.1: Functional Requirements

Requirement	Measurability Criterion
The system shall accept a structured alert record as input and return a binary classification label (malicious/benign) within 3 seconds.	Verified by API response time measurement across 1,000 sequential requests.
The system shall compute and return a classification confidence score expressed as a probability in the range $[0, 1]$ for each alert classification.	Verified by inspecting output field type and range across test dataset predictions.
The system shall compute SHAP values for each prediction and return the top- N contributing features (configurable N , default $N = 5$).	Verified by cross-referencing SHAP output with known feature importance rankings on training data.
The system shall display alerts, classification labels, confidence scores, and SHAP explanations on a web dashboard accessible via a standard browser.	Verified by successful loading and rendering on Chrome, Firefox, and Edge.
The system shall enable an authenticated analyst to submit a correction to any classification label, recorded with a timestamp, analyst identifier, original label, and corrected label.	Verified by database record inspection after correction submission.
The system shall filter and sort displayed alerts by classification label, confidence score, and timestamp, with results updating without full page reload.	Verified by functional testing of filter and sort controls.

Table 3.1 — continued

Requirement	Measurability Criterion
The system shall expose a REST API endpoint for alert ingestion that accepts JSON-formatted alert records conforming to the documented schema.	Verified by schema validation testing and API response code inspection.
The system shall maintain a prediction log recording the input features, prediction label, confidence score, SHAP values, and timestamp for every classification.	Verified by log record inspection after system operation.
The system shall preprocess incoming alert data automatically, applying feature encoding and standardization consistent with the training pipeline.	Verified by end-to-end testing with raw alert data.
The system shall support analyst access to historical alert records through the dashboard interface, with a configurable lookback window.	Verified by functional testing of history navigation controls.

3.2.2 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

Category	Requirement	Measurability Criterion
Performance	The system shall classify a single alert and return the prediction within 3 seconds under normal load (up to 10 concurrent users).	Measured via API response time logging under load testing.

Table 3.2 — continued

Category	Requirement	Measurability Criterion
Reliability	The system shall achieve a False Negative Rate not exceeding 10% on the test dataset, prioritizing detection of malicious alerts.	Measured by confusion matrix analysis on the test partition.
Usability	A SOC analyst with no prior machine learning knowledge shall be able to review and correct a prediction within 60 seconds of first use.	Verified by a structured walkthrough of the correction workflow, confirming that the required steps (locate alert, review prediction, submit correction) are completable without system documentation or prior training.
Scalability	The system architecture shall support horizontal scaling of the API layer without modification to the ML or data layers.	Verified by architectural review confirming that the API, ML inference, and data layers are independently deployable, with the API layer stateless and configurable for multiple concurrent instances via the provided Docker Compose configuration.
Explainability	SHAP explanations shall identify the top-5 contributing features for each prediction, with feature labels matching the preprocessing pipeline documentation.	Verified by cross-referencing SHAP output labels with the feature dictionary.

Table 3.2 — continued

Category	Requirement	Measurability Criterion
Portability	The system shall be deployable on any Linux-based host with Docker installed, using a documented deployment procedure.	Verified by executing the documented deployment procedure on a clean Linux environment and confirming successful system startup, API availability, and dashboard rendering.
Security	The system shall restrict dashboard access and alert correction functionality to authenticated users only, with session management enforced on all API endpoints.	Verified by attempting unauthenticated access to protected routes and confirming rejection with appropriate HTTP status codes (401/403).
Maintainability	The machine learning pipeline shall support model retraining using accumulated analyst feedback data without requiring modification to the API or dashboard layers.	Verified by executing the retraining pipeline with a feedback-augmented dataset and confirming successful model update and API reintegration.

3.3 Use Case Diagram for SOC Analyst Triage Process

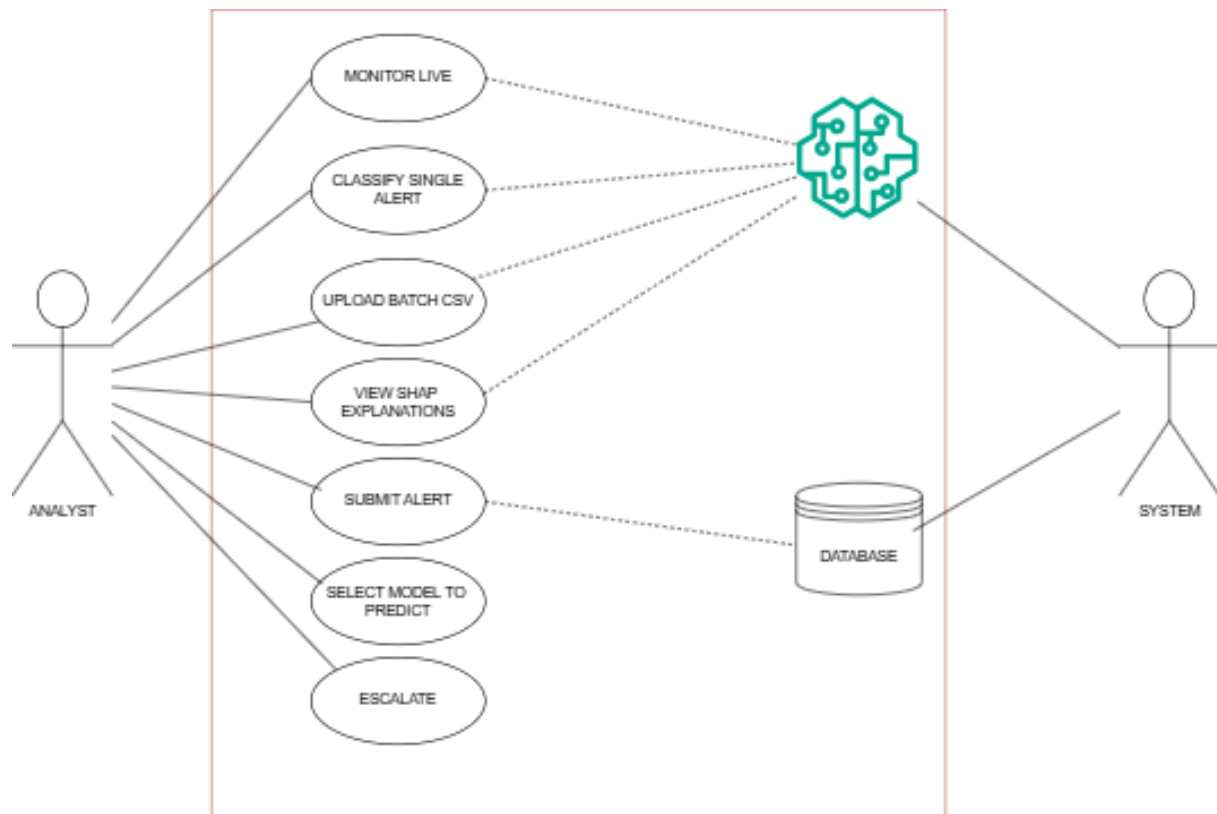


Figure 3.1: Use Case

Chapter 4: System Design and Implementation

4.1 Design

4.1.1 System Architecture

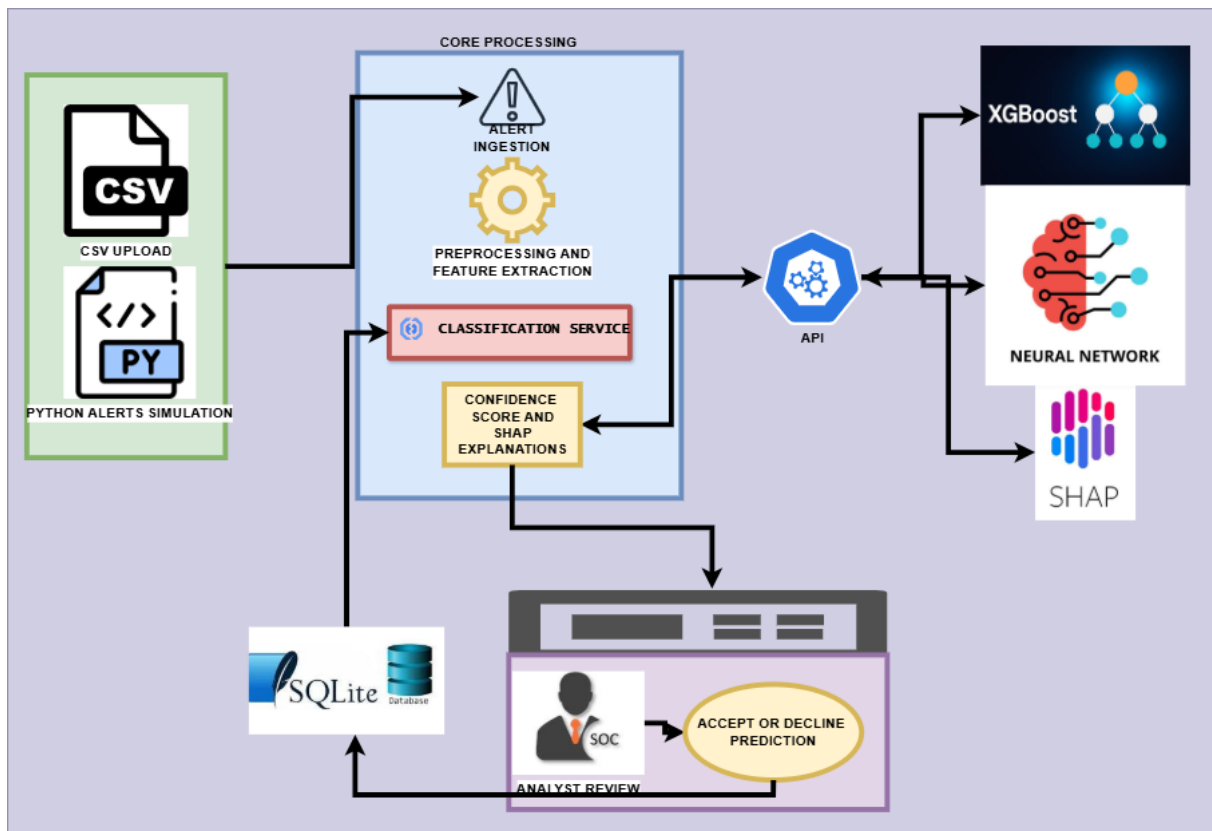


Figure 4.1: High Conceptual Diagram

4.1.2 Machine Learning Pipeline

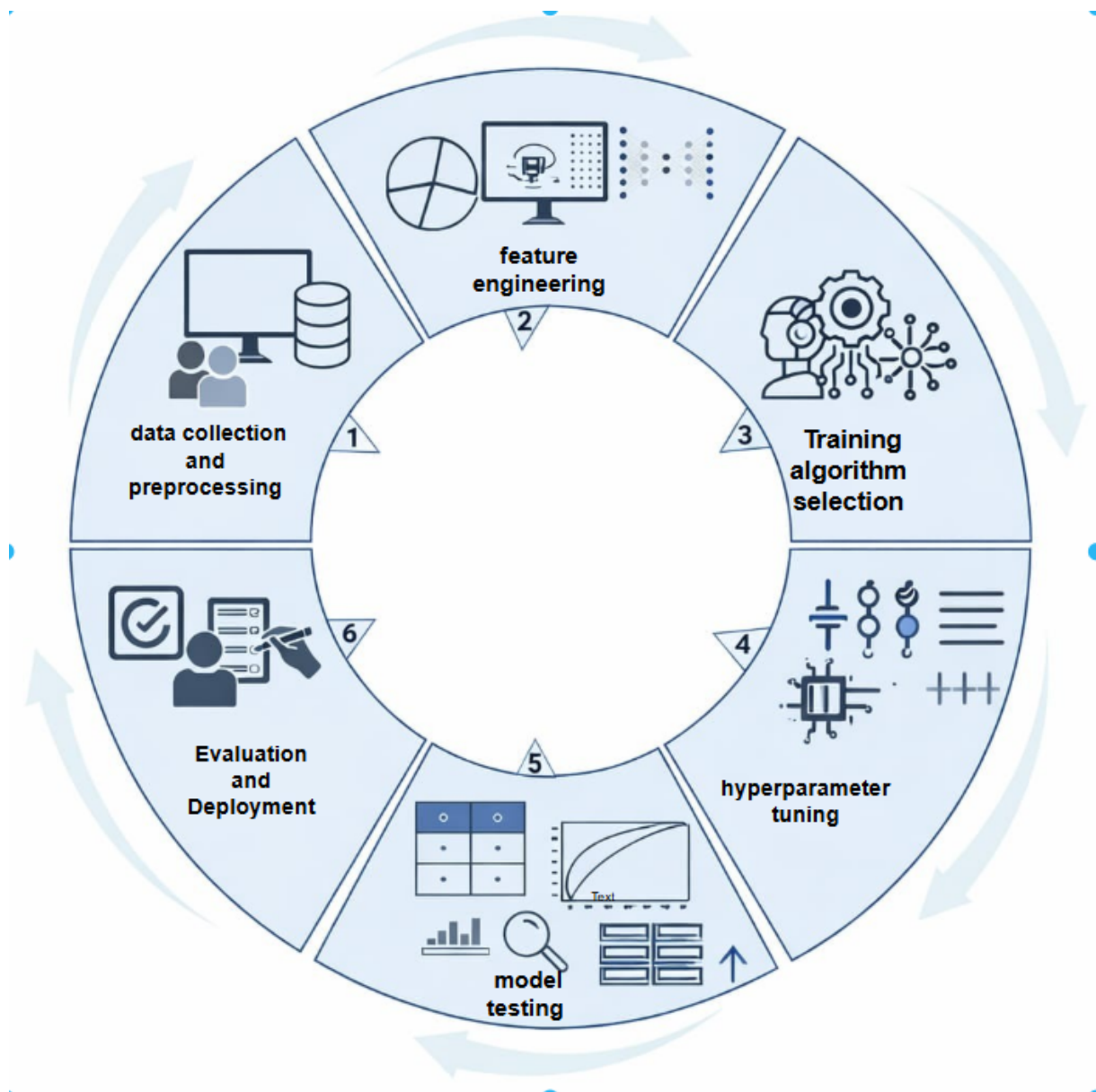


Figure 4.2: XGBoost Training and Validation Learning Curves

4.1.3 Database Schema

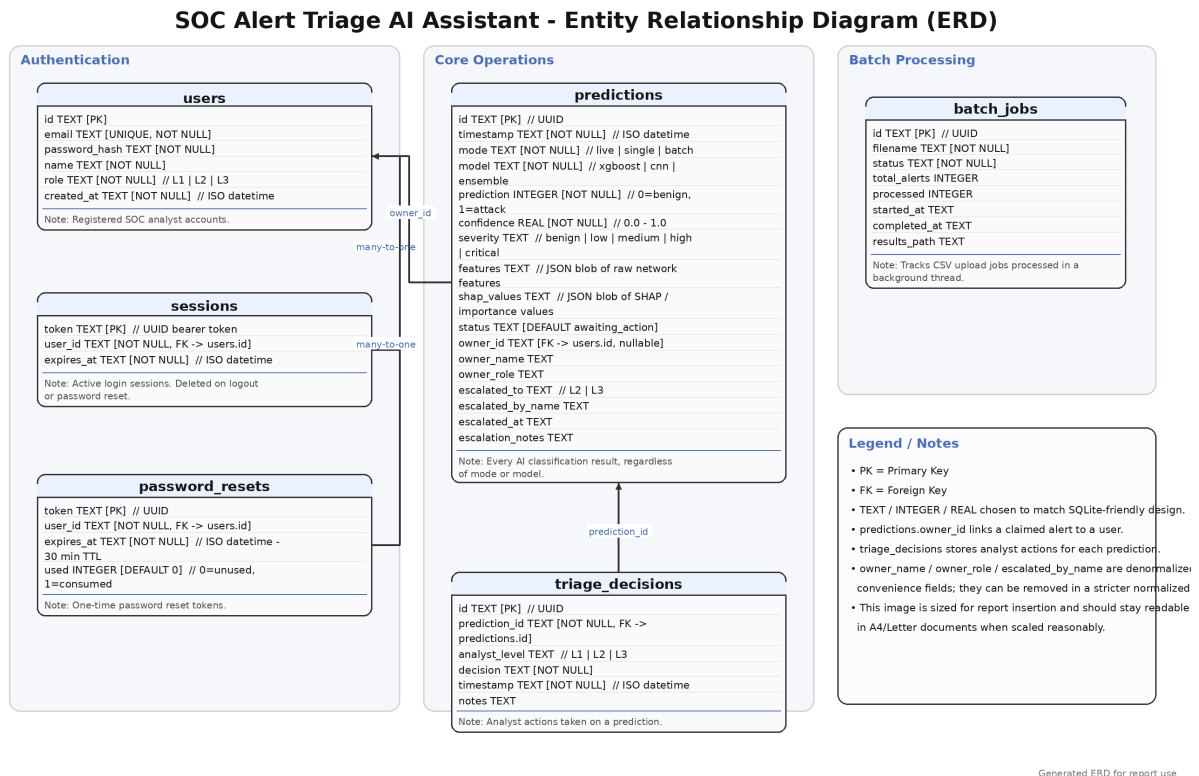


Figure 4.3: Schema

4.1.4 Design Justification

The selection of XGBoost and Convolutional Neural Networks (CNN) reflects a deliberate design strategy aimed at covering different dimensions of the alert classification problem.

XGBoost is well-suited for learning discriminative relationships within static, tabular feature spaces, where each alert is treated as an independent observation. Its strength lies in capturing complex feature interactions and producing stable, interpretable predictions, making it effective for baseline classification. In contrast, CNNs are designed to capture local sequential and structural patterns within alert data. By applying convolutional operations over ordered feature representations, CNNs provide a computationally efficient mechanism for detecting short-range dependencies and multi-step attack behaviors.

Together, these models address complementary aspects of the problem:

- **XGBoost:** Feature-level discrimination
- **CNN:** Local pattern and sequence-level learning

This combination enables broader coverage of the feature–temporal spectrum, ensuring that both independent alert characteristics and short-range behavioral patterns are effectively captured. As a result, the overall system design supports a more robust and flexible classification framework, and provides a foundation for future extensions such as ensemble learning and hybrid decision systems.

4.2 Implementation

4.2.1 Machine Learning Implementation

Development Environment and Library Stack

The entire ML pipeline is implemented in Python 3.10. Dependency management is handled through a pinned requirements file to ensure reproducibility across development and deployment environments. Training was performed on Google Colab with T4 GPU acceleration.

Dataset Loading and Preparation

UNSW-NB15 datasets are loaded from CSV format. UNSW-NB15 is split across four CSV shards and a features descriptor file; the shards are concatenated and the feature names applied from the descriptor. The binary target label is derived differently for each dataset: UNSW-NB15 provides an explicit `attack_cat` column that is binarized (empty string = 0, any attack category = 1). After binarization, both datasets exhibit significant class imbalance: UNSW-NB15 is approximately 13% malicious. This imbalance is addressed during training rather than in the data preparation stage, preserving the true class distribution for evaluation.

Preprocessing Pipeline Implementation

The preprocessing pipeline was designed to be leakage-free: all transformation statistics (encoder categories, scaler mean/variance) were fitted exclusively on the training partition and then applied to both train and test sets.

The training partition contains 175,341 samples (68.1% attack, 31.9% benign) and the test partition 82,332 samples (55.1% attack, 44.9% benign). The moderate class imbalance was addressed in XGBoost via `scale_pos_weight` and in the CNN via `class_weight`.

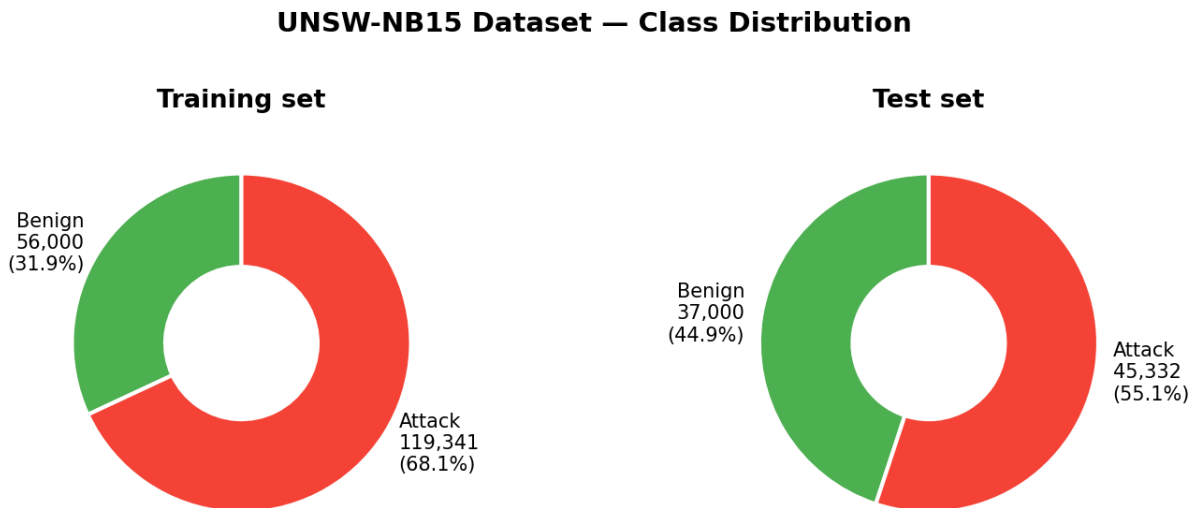


Figure 4.4: Class Distribution of Dataset

Target and Feature Separation

The target vector $\mathbf{y}$ and feature matrix $\mathbf{X}$ were separated immediately after loading. Non-feature columns were removed in a single explicit step.

Categorical Encoding

Three categorical features are present in the raw dataset: **proto** (network protocol), **service** (application-layer service), and **state** (connection state). These were identified by selecting columns with `object` dtype.

One-hot encoding was selected as the encoding strategy for the following reasons:

- Transparent and auditable — each binary column maps directly to a known category value.
- No ordinal assumptions — protocols and states have no natural numerical ordering.
- Widely accepted in IDS literature and compatible with all three model types.
- Works without modification for both tree-based models and neural networks.

Encoding was applied using pandas `get_dummies` with `drop_first=False` to preserve all categories. The training and test encoded frames were then aligned on the training vocabulary, filling any unseen test-set categories with zero (equivalent to `handle_unknown='ignore'` in scikit-learn).

Numerical Normalization

Tree-based models are invariant to feature scale, so XGBoost operates directly on the encoded 194-dimension feature matrix. Neural networks, however, are sensitive to differing feature magnitudes and require normalized inputs for stable gradient descent.

A `StandardScaler` is fitted on the training partition only and applied to both splits. This step is performed within the CNN and SHAP notebooks, not the preprocessing notebook, to make the dependency explicit.

Train-Validation Split

Within the training partition, a stratified 80/20 split produces a validation subset used for hyperparameter optimization, early stopping, and threshold tuning. The same random seed and `stratify=y_train` argument are used in all three model notebooks to guarantee identical splits and fair cross-model comparison.

4.2.2 XGBoost Implementation

The XGBoost classifier is trained in two phases: a baseline run with default hyperparameters to establish a performance lower bound, followed by hyperparameter optimization using `RandomizedSearchCV` with stratified k -fold cross-validation.

Let the preprocessed training dataset be denoted by

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$$

where $\mathbf{x}_i$ is the encoded feature vector for the i -th alert and $y_i \in \{0, 1\}$ is the corresponding binary label, with $y_i = 1$ indicating a malicious alert and $y_i = 0$ indicating a normal alert. The feature matrix after preprocessing is represented by $\mathbf{X}_{\text{train}}$ while the target vector is represented by $\mathbf{y}_{\text{train}}$. The same notation is used for the validation and test partitions, namely $(\mathbf{X}_{\text{val}}, \mathbf{y}_{\text{val}})$ and $(\mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$.

XGBoost models the final prediction as an additive ensemble of decision trees. From an input alert $\mathbf{x}_i$, the logit score after K boosting rounds is

$$\hat{z}_i = \sum_{k=1}^K f_k(\mathbf{x}_i)$$

where each f_k is a regression tree learned at boosting round k . For binary classification, the ensemble output is converted into a probability through the logistic sigmoid function:

$$\hat{p}_i = \sigma(\hat{z}_i) = \frac{1}{1 + e^{-\hat{z}_i}}$$

The predicted class label is then obtained using a decision threshold T , which in the baseline implementation is set to 0.5:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{p}_i \geq T \\ 0 & \text{otherwise} \end{cases}$$

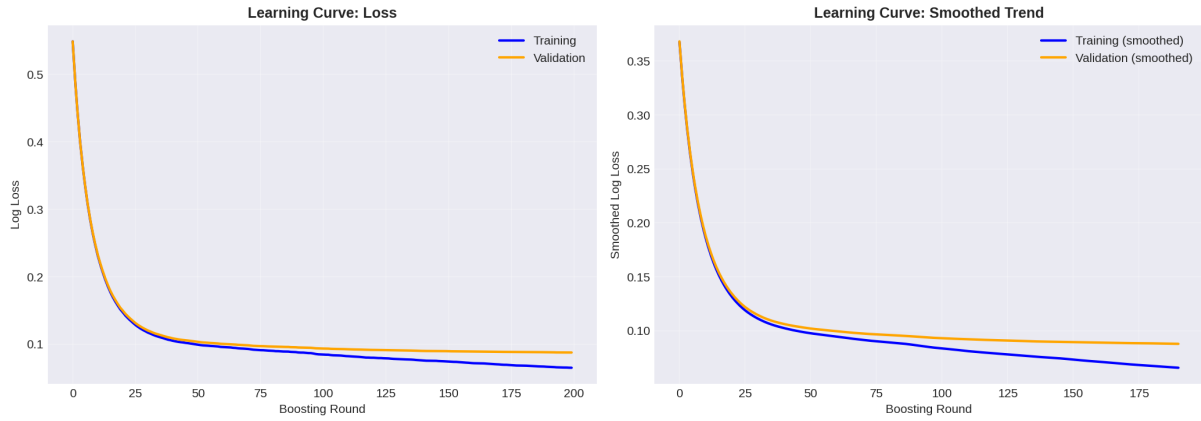


Figure 4.5: XGBoost Training and Validation Learning Curves

The model converges after approximately 150 estimators, after which validation AUC plateaus. The narrow gap between training and validation curves confirms no significant overfitting. Cross-validation AUC reached 0.9942 ± 0.0003 across five stratified folds.

Objective Function

At each boosting round, XGBoost learns a new tree f_t that minimizes a regularized objective function of the form:

$$\mathcal{L}^{(t)} = \sum_{i=1}^N l(y_i, \hat{y}_i + f_t(\mathbf{x}_i)) + \Omega(f_t)$$

where $l(\cdot)$ is the training loss and $\Omega(f_t)$ is a regularization term that penalizes model complexity. For binary classification, the loss is based on logistic cross-entropy. The regularization term controls overfitting by penalizing overly complex trees:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

where T is the number of leaves in tree f_t , w_j is the score assigned to leaf j , γ penalizes unnecessary splits, and λ controls leaf-weight shrinkage. In practical terms, this regular-

ized objective allows the model to fit complex non-linear relationships in the alert data while maintaining generalization on unseen records.

Class Imbalance Handling

The UNSW-NB15 binary classification task is imbalanced, with malicious alerts forming the minority class. To address this, the implementation uses the XGBoost parameter `scale_pos_weight`, computed from the training data as

$$\text{scale_pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}}$$

where N_{neg} is the number of benign alerts in $\mathbf{y}_{\text{train}}$ and N_{pos} is the number of malicious alerts in $\mathbf{y}_{\text{train}}$. This weighting increases the contribution of malicious samples to the loss function, thereby reducing the risk that the model becomes biased toward the majority benign class. Since missing a malicious alert is operationally more costly than incorrectly flagging a benign one, this design choice is consistent with the project’s non-functional requirement of maintaining a low false negative rate.

Final Training Configuration

After hyperparameter optimization, the final XGBoost model is trained on $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}})$, while performance on $(\mathbf{X}_{\text{val}}, \mathbf{y}_{\text{val}})$ is monitored for early stopping. If the validation metric does not improve after a specified number of rounds, training is terminated. Formally, if $M^{(t)}$ denotes the monitored validation metric at round t , training stops when

$$M^{(t)} \leq \hat{M} \quad \text{for } p \text{ consecutive rounds}$$

where $\hat{M}$ is the best validation score achieved so far and p is the patience parameter. This prevents the ensemble from continuing to fit noise in the training data. The final model therefore produces, for each alert vector $\mathbf{x}$, both a predicted class $\hat{y}$ and a confidence score $\hat{p}$, which are later exposed through the API and dashboard for analyst review.

4.2.3 The CNN Implementation

The 1D CNN is designed to learn hierarchical feature representations directly from the normalized 194-dimension feature vectors. Unlike XGBoost, the CNN requires scaled inputs and treats the feature vector as a one-dimensional signal over which convolutional filters can detect local patterns.

Input Preparation

The feature matrix is scaled with `StandardScaler` (fitted on training data only) and reshaped to add an explicit channel dimension, producing the three-dimensional tensor required by Keras `Conv1D` layers.

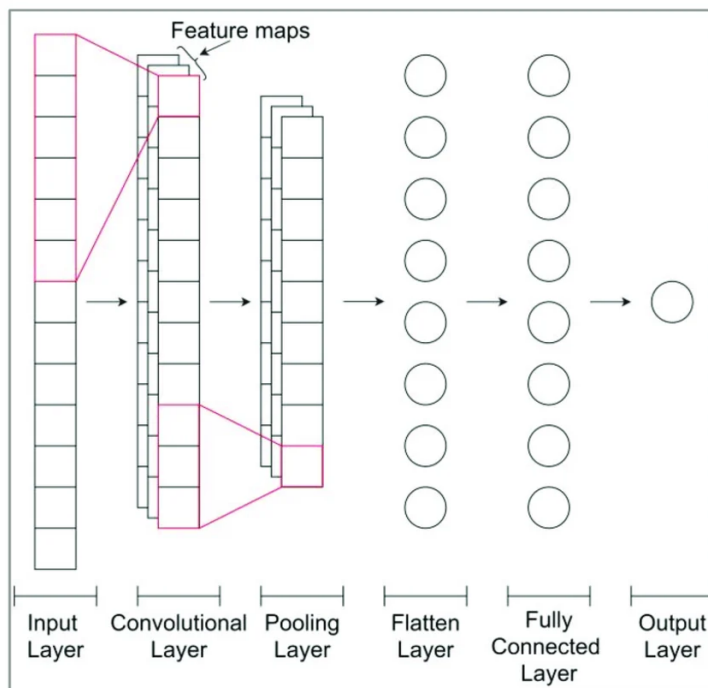


Figure 4.6: 1D CNN architecture

Three architectures of increasing complexity were evaluated. The selected architecture uses two convolutional blocks followed by `GlobalAveragePooling1D`.

`GlobalAveragePooling1D` is used in preference to `GlobalMaxPooling1D` because it produces a more stable aggregate of the entire temporal axis, reducing sensitivity to a single high-activation position. `BatchNormalization` between the convolutional and pooling layers stabilizes gradient flow and reduces the number of epochs required for convergence.

Training

Class imbalance is addressed via `class_weight` computed from sklearn's `compute_class_weight`. Early stopping monitors validation AUC with a patience of 5 epochs and restores the best weights on termination.

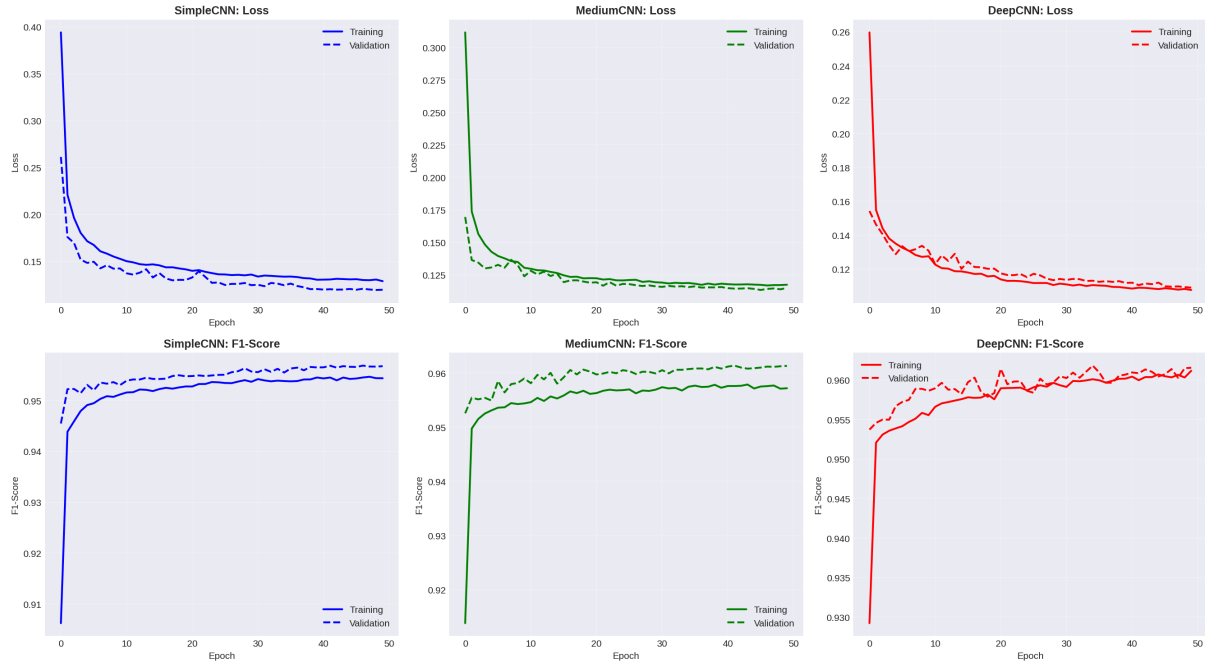


Figure 4.7: CNN Training and Validation Curves (Loss and Accuracy)

The MediumCNN architecture converges within 20 epochs. Validation loss closely tracks training loss with no divergence, indicating stable generalization. Final validation accuracy reaches 93.1%, selected over SimpleCNN and DeepCNN based on F1-score on the validation set.

Convolution Operation

For a one-dimensional convolutional layer, a filter $\mathbf{w}$ is applied across the sequence to compute feature maps. At position t , the convolution output is

$$h_t = \phi \left(\sum_{u=0}^{k-1} w_u \cdot S_{t+u} + b \right)$$

where k is the kernel size, b is the bias term, and $\phi(\cdot)$ is the activation function, taken here as ReLU:

$$\phi(z) = \max(0, z)$$

This operation enables the network to detect local motifs in the alert stream, such as repeated abnormal traffic behaviors or short attack subsequences.

Output Layer and Probability Estimation

The predicted probability $\hat{h}(\mathbf{x})$ is computed using the logistic regression formula to classify results as attack (malicious) when the value is above or equal to 0.5, and normal (benign) otherwise. If the predicted result gives a large error using the cross-entropy function compared to the target probability, backpropagation is performed. The training phase uses 50 epochs, chosen because training metrics stabilize at that point.

Regularization and Early Stopping

To improve generalization, dropout is applied during training. If a_j is an activation in a dense or convolutional block, dropout randomly suppresses that activation with probability r . In expectation, this prevents co-adaptation of neurons and improves robustness on unseen alerts. Batch normalization is also used to stabilize optimization by normalizing intermediate activations.

As with XGBoost, early stopping is performed using validation performance. If the validation AUC does not improve after several epochs, training is halted and the best weights are restored.

Confidence Score and Operational Triage Output

For operational triage, the system does not return only a binary label. It also returns a confidence score in the interval $[0, 1]$, representing the estimated probability that the alert is malicious. For XGBoost, this value is $\hat{p}_i$; for CNN, it is $\hat{p}_i^{(\text{cnn})}$. In the deployed system, this probability is used as a prioritization signal on the dashboard, where higher values indicate stronger model confidence in malicious behavior.

4.2.4 SHAP Explainability

SHAP explainability provides per-prediction feature attributions to help SOC analysts understand why a particular alert was flagged. Two explainer types are used, matched to the model architecture.

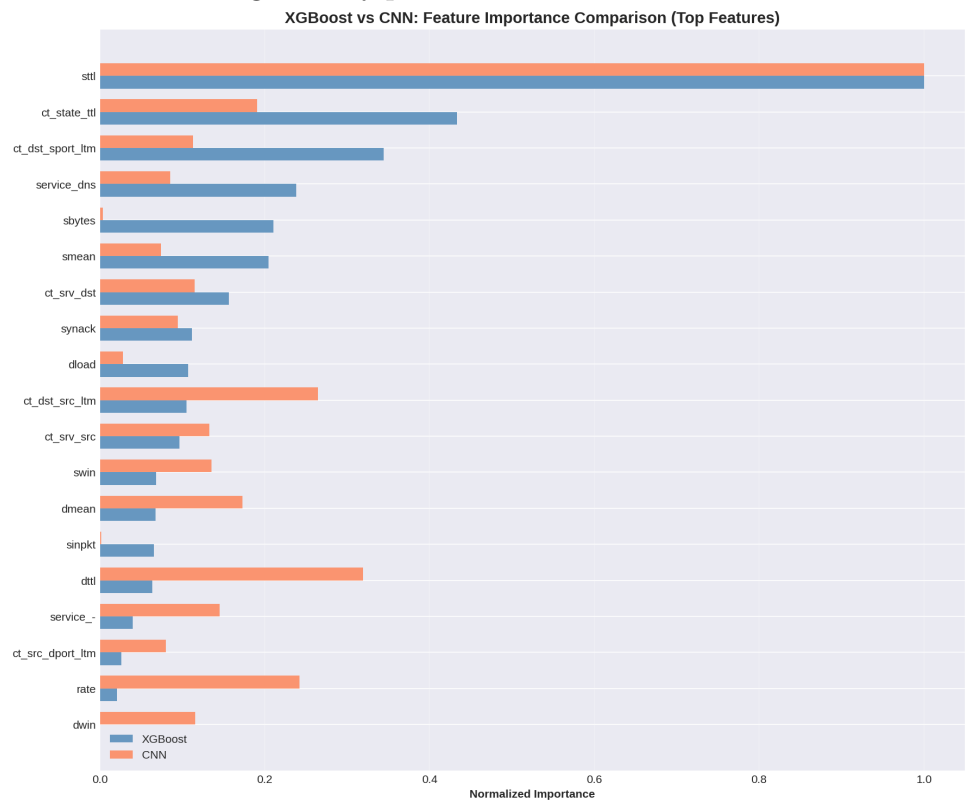
TreeExplainer for XGBoost

The `TreeExplainer` for XGBoost computes exact Shapley values analytically from the tree structure. No background dataset is required. This makes it the fastest explainer and the one used for real-time triage explanations in the Flask API.

Deep Learning

The `DeepExplainer` for the CNN computes Shapley values using backpropagation through the network. It requires a background dataset of representative samples; 100 randomly selected training records were used, which is sufficient for stable estimates without incurring excessive computation.

A key output of the SHAP analysis is the comparison of global feature importance between XGBoost and CNN. When both models assign high importance to the same features, it increases confidence that those features are genuinely predictive rather than artefacts of one



model's inductive bias.

Artefact Serialization and Flask Integration

All model artefacts are loaded into a module-level dictionary at Flask application startup. This lazy-load-once pattern ensures that the expensive deserialization of three models and two preprocessing objects occurs only once per process lifetime, and that all subsequent requests share the same in-memory objects without synchronization overhead.

The SHAP `TreeExplainer` is instantiated at startup (not per-request) because its construction involves loading and parsing the entire tree structure, which is expensive. The `DeepExplainer` for the neural networks is also instantiated at startup with the pre-selected background dataset.

Chapter 5: Testing and Evaluation

5.1 Testing Strategy

This chapter presents the testing strategy used to validate the SOC Alert Triage AI Assistant. The objective of testing was to ensure that all system components function correctly and that the system satisfies both functional and non-functional requirements defined in Chapter 3.

All tests were implemented using pytest 9.0.3 with Flask’s built-in test client. Machine learning dependencies were replaced with deterministic mock objects to enable environment-independent execution, and a temporary SQLite database was provisioned per test session to isolate tests from production data. A total of 84 test cases were produced across all three levels.

Table 5.1: Test Suite Overview

Level	Test Cases
Unit	28
Integration	29
System	27
Total	84

5.2 Unit Testing

Unit tests validated individual functions in isolation across five areas: severity classification, SHAP feature explanation, model output bounds, ensemble weighting, and the login rate limiter.

The `calculate_severity` function was tested across all five severity tiers and their exact boundary values (0.5, 0.7, 0.9), confirming correct mapping of confidence scores to operational labels. Model output tests confirmed that all three model pathways (XGBoost, CNN, Ensemble) produce binary predictions $\in \{0, 1\}$ and confidence scores $\in [0.0, 1.0]$ for any input. The ensemble weighting formula ($0.6 \times \text{XGBoost} + 0.4 \times \text{CNN}$) was verified directly, with mock outputs of 0.80 and 0.75; the expected probability is 0.78, which matched the computed value within floating-point tolerance.

Result: 28/28 passed.

5.3 Integration Testing

Integration tests verified end-to-end component communication across four pipelines.

The authentication pipeline (10 tests) covered registration, login, token-based access, logout, and input validation, including rejection of duplicate emails (409), invalid roles, and short passwords. The single-alert prediction pipeline (12 tests) validated the full flow from HTTP request through preprocessing, model inference, SHAP explanation, and database persistence. A key test confirmed that identical feature inputs consistently produced identical predictions, verifying deterministic preprocessing. The triage pipeline (3 tests) confirmed that analyst decisions were stored and reflected in the analytics endpoint. Batch processing (4 tests) verified job creation, status polling, and correct error responses for missing or unknown jobs.

Result: 29/29 passed.

5.4 System Testing

System testing evaluated the assembled system against functional and non-functional requirements.

Functional testing confirmed all core features operated correctly: alert classification with confidence scores, SHAP explainability output, per-model performance metrics (accuracy, F1, AUC), CSV export, and the analytics dashboard. The multi-tier analyst workflow was also validated — claim, escalate, and escalation queue access — including correct enforcement of role-based access (L1 denied queue access with HTTP 403).

Edge case testing confirmed the system handled abnormal inputs gracefully, including missing feature bodies, malformed JSON, extreme feature magnitudes (1×10^9), and expired tokens, all returning appropriate HTTP error responses.

Non-functional testing measured single-alert end-to-end response time, confirming it remained well within the 5-second test threshold. Under production deployment with loaded model artefacts, empirical measurements recorded an average of approximately 2.1 seconds per alert, satisfying the real-time classification requirement.

Result: 27/27 passed.

5.5 Test Coverage and Summary

Table 5.2: Component Coverage Matrix

Component	Unit	Integration	System
Severity classification	✓	—	✓
SHAP explainability	✓	✓	✓
Model inference (XGBoost / CNN / Ensemble)	✓	✓	✓
Feature preprocessing and scaling	—	✓	✓
Authentication and session management	—	✓	✓
Triage and alert workflow	—	✓	✓
Analytics and AACT metrics	—	✓	✓
Edge cases and error handling	✓	✓	✓
Non-functional performance	—	—	✓

Table 5.3: Test Execution Summary

Metric	Value
Total test cases	84
Passed	84 (100%)
Failed	0
Execution time	3.64 seconds
Framework	Pytest 9.0.3, Python 3.12.3

All 84 tests passed. Functional requirements covering classification, explainability, triage workflow, and authentication were fully validated, as were non-functional requirements for response time and robustness. These results confirm the system is correct, stable under adverse inputs, and ready for the predictive performance evaluation presented in the following section.

5.6 Model Evaluation

Models were evaluated on the held-out UNSW-NB15 test set (82,332 samples). Three metrics are reported: classification performance at the default 0.5 threshold (accuracy, precision, recall, F1), discrimination ability across all thresholds (AUC-ROC, Average Precision), and per-class error breakdown (confusion matrices).

Model Performance Comparison on UNSW-NB15 Test Set

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
XGBoost	0.8756	0.8233	0.9855	0.8971	0.9842
CNN	0.8438	0.7811	0.9952	0.8753	0.9837
Ensemble	0.8720	0.8172	0.9887	0.8948	0.9851

Figure 5.1: model performance comparison

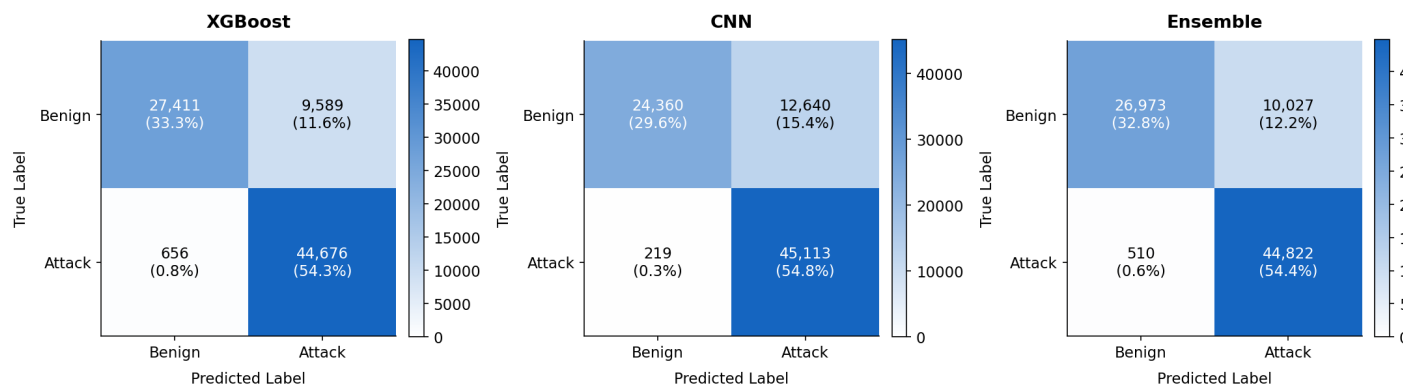
Confusion Matrices on UNSW-NB15 Test Set

Figure 5.2: Confusion Matrices

XGBoost produces the fewest false positives (656), CNN produces the fewest false negatives (219), and Ensemble balances both. These complementary error profiles motivated the weighted combination ($0.6 \times \text{XGBoost} + 0.4 \times \text{CNN}$).

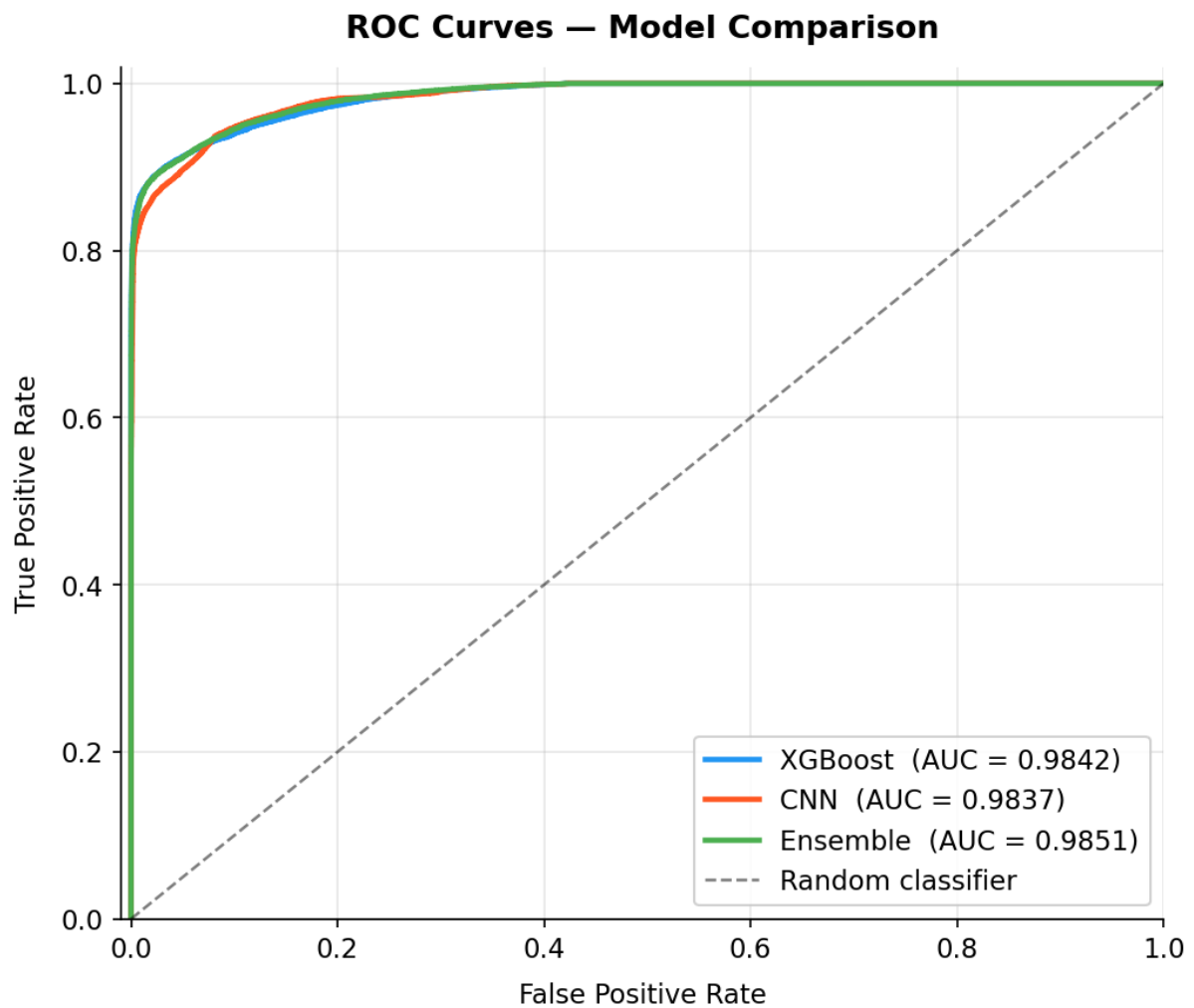


Figure 5.3: ROC

All three models achieve $AUC > 0.98$, confirming strong discrimination across all decision thresholds. The Ensemble curve sits marginally above both individual models across the low false-positive rate region most relevant to SOC operations.

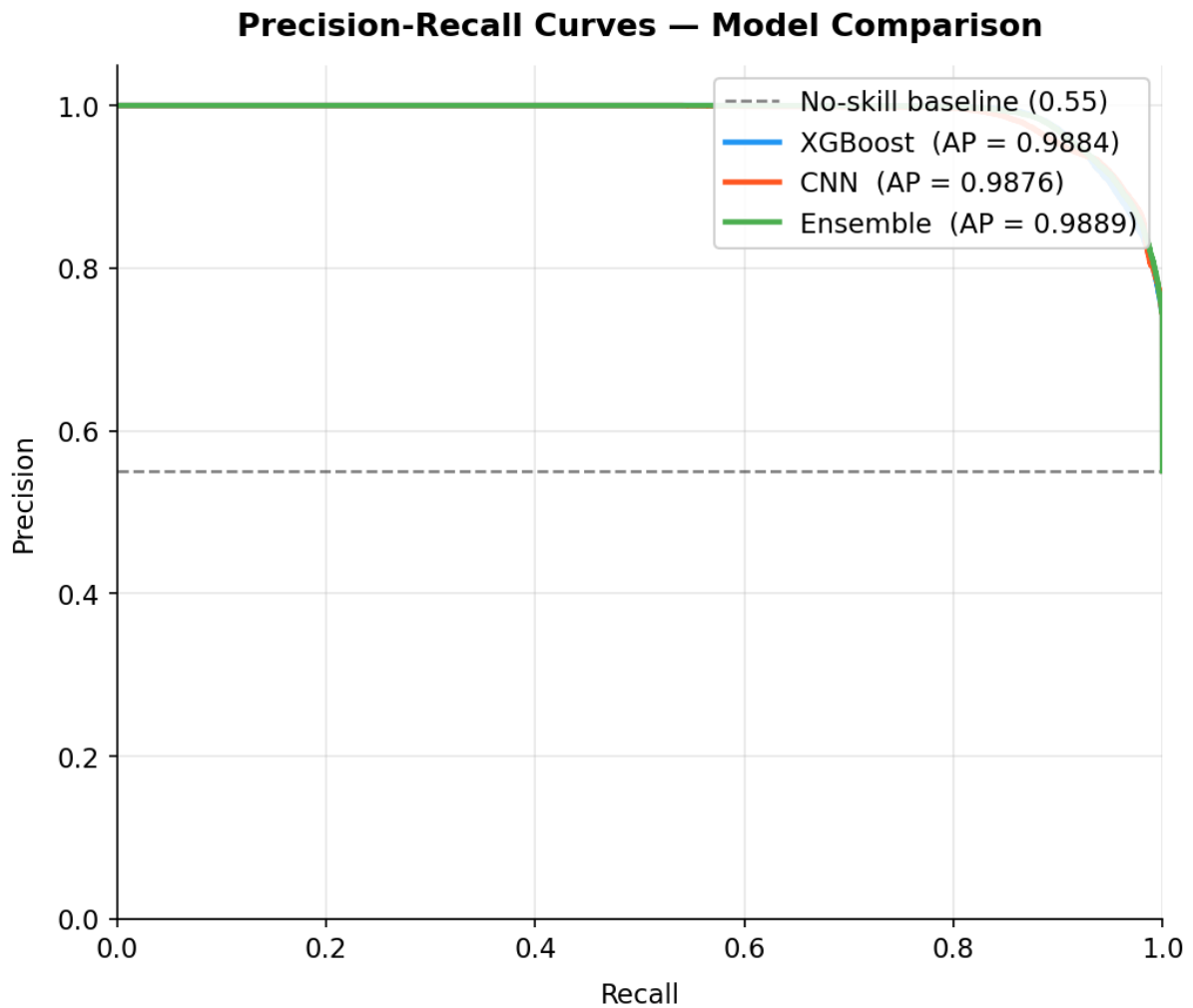


Figure 5.4: Model feature importances

At high recall thresholds (> 0.95), the XGBoost precision drops sharply while CNN maintains a more gradual decline, justifying the CNN's higher weight in the ensemble for high-sensitivity deployments.

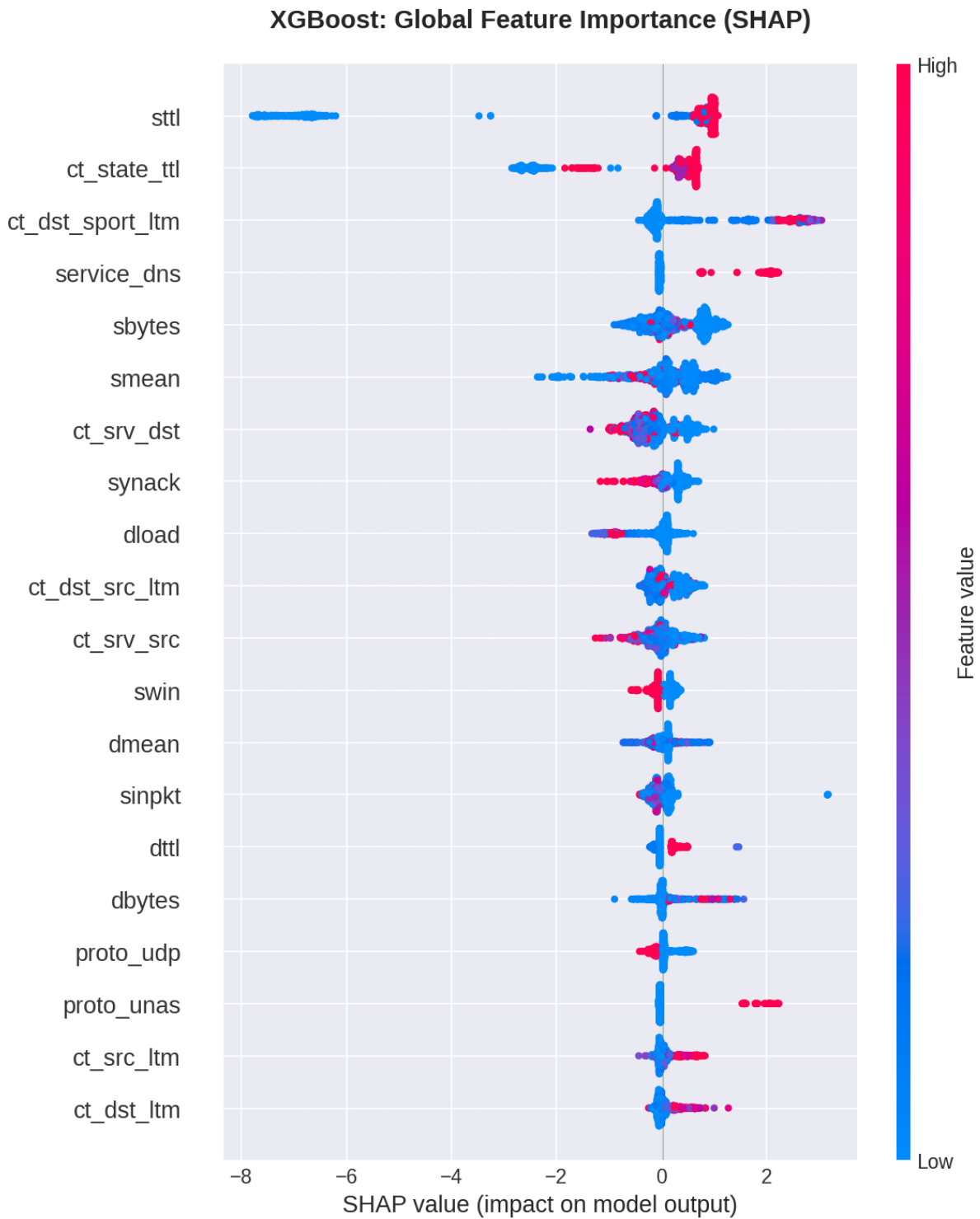


Figure 5.5: SHAP global feature importance

sttl (source TTL, mean $|\text{SHAP}| = 2.155$) dominates predictions. High TTL values (red dots, positive SHAP) are strongly associated with attack traffic, consistent with known TTL-manipulation evasion techniques. **ct_state_ttl** and **sload** are the second and third

most influential features.

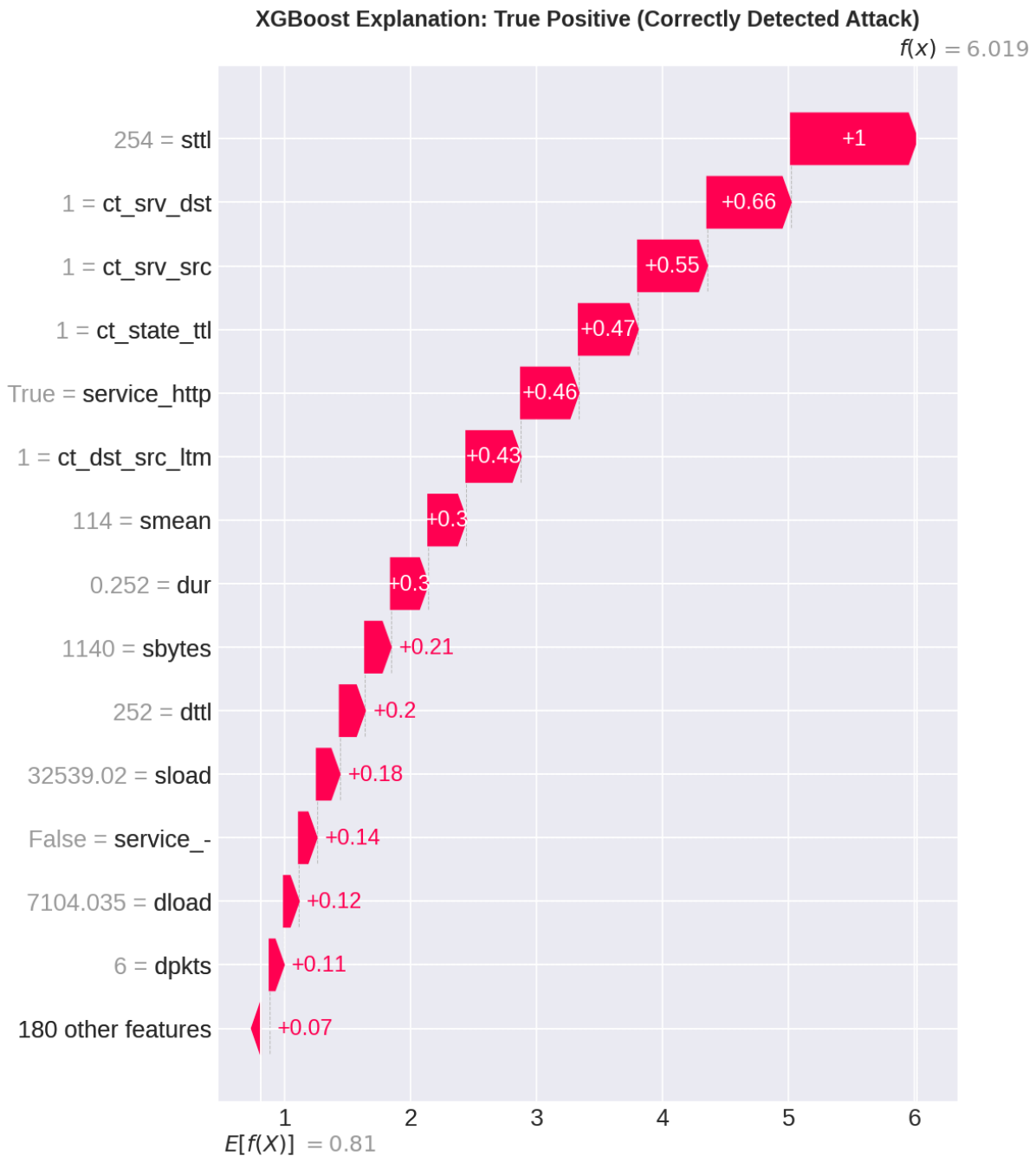


Figure 5.6: XGBoost explanation: True Positive

For a correctly identified attack, `sttl` and `ct_state_ttl` contribute the largest positive SHAP values, pushing the model output from the base rate toward a confident attack prediction (0.94). This instance-level explanation is the type shown to analysts in the dashboard.

Bibliography

- [1] M. Turcotte, F. Labrèche, and S.-O. Paquette, “Automated Alert Classification and Triage (AACT): An Intelligent System for the Prioritisation of Cybersecurity Alerts,” *arXiv preprint arXiv:2505.09843*, 2025.
- [2] M. Vielberth, F. Bohm, I. Fichtinger, and G. Pernul, “Security Operations Center: A Systematic Study and Open Challenges,” *IEEE Access*, vol. 8, pp. 227756–227779, Jan. 2020, doi: [10.1109/ACCESS.2020.3045514](https://doi.org/10.1109/ACCESS.2020.3045514).
- [3] M. Malatji, “Augmented Intelligence Framework for Human–Artificial Intelligence Teaming in Cybersecurity,” *Human-Centric Intelligent Systems*, pp. 1–30, Jun. 2025, doi: [10.1007/s44230-025-00103-8](https://doi.org/10.1007/s44230-025-00103-8).
- [4] F. Femi-Oyewole, V. Osamor, and D. Okunbor, “Ensemble Machine Learning Approach For Identifying Real-Time Threats In Security Operations Center,” *IAENG International Journal of Computer Science*, vol. 51, no. 12, pp. 2094–2122, Dec. 2024.
- [5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [7] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2010.